

SPRAWOZDANIE SZTUCZNA INTELIGENCJA

Celem tych laboratoriów jest implementacja algorytmu bfs i dfs w celu znalezienia rozwiązania dla zadania n-hetmanów. W celu wykonania laboratoriów posłużyłem się wcześniej napisaną funkcją conflicts (na laboratoriach 1) oraz użyłem dwóch reprezentacji board_array i board_perm.

Modyfikacja funkcji conflicts

```
def conflicts(board): 4 usages
    my_array=np.array(board)
    n=my_array.shape[0]
    help_array = [-1] * n
    if my_array.ndim == 2:
        for r in range(n):
            for c in range(n):
                if my_array[r][c] == 1:
                    if help_array[r] != -1 or c in help_array:
                        return False
                    help_array[r]=c
            if -1 in help_array:
                return False
    else:
        if -1 in my_array:
            return False
        help_array=my_array
    for i in range(n):
        for j in range(i+1,n):
            if abs(help_array[i]-help_array[j]) == abs(i-j):
                return False
    return True
```

MODYFIKACJA 1

W celu poprawy wydajności i elastyczności zmodyfikowałem funkcję conflicts z ostatniego zadania. Wcześniej funkcja obsługiwała cztery reprezentacje i najpierw modyfikowała struktury do board_vector a następnie do board_perm i na samy końcu sprawdzało konflikty na przekątnej. Teraz funkcja obsługuje tylko board_array i board_perm. Board_array modyfikuje do board_perm. Reprezentacja board_perm nie

jest modyfikowana. Na samym końcu struktury są sprawdzane pod względem konfliktów na przekątnych.

MODYFIKACJA 2

Funkcja `conflicts` działała tylko wtedy kiedy struktury były wypełnione `n` hetmanami. Natomiast funkcja `bfs` i `dfs` potrzebuje sprawdzenia również tablic zawierające mniej niż `n` hetmanów. W celu większej elastyczności dodaje sprawdzenie czy w przekształconej `board_array` nie ma wartości `-1` (brak hetmana) oraz dla `board_perm` czy tablica wprowadzona do funkcji nie posiada także `-1` (brak hetmana/hetmanów). Jeśli występują braki funkcja zwraca `false`

GENEROWANIE STANÓW POTOMNYCH

```
def gen_succesors(board,n,closed,open,type,depth): 4 usages
    if type=="board_array":
        for i in range(n):
            for j in range(n):
                new_board = [row[:] for row in board]
                if board[i][j] == 0:
                    new_board[i][j] = 1
                    key = tuple(tuple(row) for row in new_board)
                    if (key, depth + 1) not in closed and (new_board, depth + 1) not in open:
                        open.append((new_board, depth + 1))
        return open
    else:
        for i in range(n):
            for j in range(n):
                new_board = board[:]
                if board[i] == -1 and j not in board:
                    new_board[i] = j
                    key = tuple(new_board)
                    if (key, depth + 1) not in closed and (new_board, depth + 1) not in open:
                        open.append((new_board, depth + 1))
        return open
```

Do funkcji wrzucamy daną tablicę, reprezentację, długość tablicy, zbiór `open` (stany do odwiedzenia), zbiór `closed` (stany odwiedzone) oraz głębokość (ilość hetmanów na planszy).

Dla `board_array` przechodzimy po całej tablicy i dla każdego pola, w którym znajduje się wartość `0`, tworzymy nową tablicę na bazie starej, w której w tym miejscu wstawiamy `1` (dodajemy nowego hetmana). Następnie sprawdzamy czy nie występuje taki przypadek w zbiorze stanów odwiedzonych i nie odwiedzonych. Oprócz takiej tablicy do zbioru wrzucamy ilość hetmanów (`depth`) w celu kontroli ile hetmanów będzie dodanych. (`Depth` zostanie bardziej objaśnione przy tłumaczeniu `dfs`).

Dla board_perm analogicznie sprawdzamy, czy w danej kolumnie nie ma już hetmana oraz dodatkowo upewniamy się, że po dodaniu nowego hetmana nie wystąpi konflikt z pozostałymi hetmanami, czyli nowy hetman nie znajduje się w tym samym wierszu co inny hetman (Brak wystąpienia j w tablicy). Jest to zasada tej struktury brak wystąpień konfliktów w wierszach i kolumnach.

FUNKCJA DFS

```
def dfs(n,name): 3 usages
    if name=="board_array":
        my_array = [[0] * n for _ in range(n)]
        closed= set()
        open=[(my_array,0)]
        while open:
            my_array, depth=open.pop()
            no_conflict=conflicts(my_array)
            if no_conflict == True:
                key_closed = tuple(tuple(row) for row in my_array)
                closed.add((key_closed,depth))
                return my_array, open, closed
            elif no_conflict ==False and depth<n:
                open = gen_succesors(my_array, n, closed, open, type: "board_array", depth)
                key_closed = tuple(tuple(row) for row in my_array)
                closed.add((key_closed, depth))

    elif name == "board_perm":
        my_array = [-1] * n
        closed = set()
        open = [(my_array,0)]
        while open:
            my_array,depth=open.pop()
            no_conflict=conflicts(my_array)
            if no_conflict == True:
                key_closed = tuple(my_array)
                closed.add(key_closed)
                return my_array, open, closed
            elif no_conflict ==False and depth<n:
                open = gen_succesors(my_array, n, closed, open, type: "board_perm", depth)
                key_closed = tuple(my_array)
                closed.add((key_closed,depth))
```

Do funkcji wrzucana jest wielkość tablicy $n \times n$ oraz typ reprezentacji. Zbiór closed implementujemy pusty typu set. Oraz zbiór open inicjalizujemy tablicą bez hetmanów. Później wykonujemy pętlę dopóki zbiór open nie będzie pusty. Pobieramy ostatni element ze stosu open sprawdzamy czy nie wystąpił konflikt jak nie wystąpił zwracamy rozwiązanie i dwa zbiory. Jak wystąpił i ilość hetmanów nie przekracza możliwej ilości. W tym celu potrzebujemy zmiennej depth. Generujemy potomków i jeśli potomkowie

nie wystąpili w zbiorze open i closed wrzucamy ich do open. Bieżący stan wrzucamy do zbioru closed. Aby było to możliwe, konwertujemy go na typ tuple, ponieważ zbiór set może przechowywać jedynie elementy gdzie wartości takiego obiektu nie można zmienić po jego utworzeniu. Funkcja ta występuje w dwóch możliwościach dla board_array i board_perm.

FUNKCJA BFS

Funkcja bfs różni jedynie się tym, że inicjalizujemy ją jak kolejkę a nie stos. Czyli pobieramy pierwszy element nie ostatni. Co powoduje, że odwiedzamy stany wszerek. Najpierw wszystkie na tej samej głębokości.

FUNKCJA EXPERIMENT

Typowa funkcja która dla sprawdza czas wykonania się danego algorytmu. Sprawdza rozwiązania dla tablic od wielkości 4x4 do wielkości 6x6. Experiment zapisuje wyniki do result.csv.

Wyniki dla różnych rozmiarów

result						
N	representation	algorithm	solution	open_count	closed_count	time
4	board_perm	bfs	[1, 3, 0, 2]	186	13	0.0009889579
4	board_perm	dfs	[1, 3, 0, 2]	43	23	0.0001966670
4	board_array	bfs	[[0, 1, 0, 0], [0, 0, 0, 1], [1, 0, 0, 0], [0, 0, 1, 0]]	698	1079	0.2049417089
4	board_array	dfs	[[0, 1, 0, 0], [0, 0, 0, 1], [1, 0, 0, 0], [0, 0, 1, 0]]	186	30	0.0078960829
5	board_perm	bfs	[0, 2, 4, 1, 3]	1427	109	0.0333004170
5	board_perm	dfs	[1, 3, 0, 2, 4]	44	47	0.0002828750
5	board_array	bfs	[[1, 0, 0, 0, 0], [0, 0, 1, 0, 0], [0, 0, 0, 0, 1], [0, 1, 0, 0, 0], [0, 0, 0, 1, 0]]	15277	45033	243.5712673749
5	board_array	dfs	[[0, 1, 0, 0, 0], [0, 0, 0, 1, 0], [1, 0, 0, 0, 0], [0, 0, 1, 0, 0], [0, 0, 0, 0, 1]]	1431	60	0.1494937091
6	board_perm	bfs	[1, 3, 5, 0, 2, 4]	12608	533	2.5958150830
6	board_perm	dfs	[1, 3, 5, 0, 2, 4]	1887	79	0.0118995421
6	board_array	Bfs	Nierozstrzygnięto	X	X	X
6	board_array	dfs	[[0, 1, 0, 0, 0, 0], [0, 0, 0, 1, 0, 0], [0, 0, 0, 0, 0, 1], [1, 0, 0, 0, 0, 0], [0, 0, 1, 0, 0, 0], [0, 0, 0, 0, 1, 0]]	93444	105	17.7233322080

Wyniki dla board_array z użyciem bfs dla planszy 6x6 zostały nierozstrzygnięte z powodu zbyt długiego czasu oczekiwania na rezultat. Wyniki wskazują znaczną przewagę w prędkości dla metody bfs. Reprezentacja planszy też ma wielkie znaczenie. Budowa board_perm oraz braku hetmanów w tych samych wierszach i kolumnach spowodowało znajdowanie szybko rozwiązania. Metoda bfs z użyciem board_perm jest najefektywniejszą metodą.

